

Avanzamento pipeline topologica — stato attuale

Obiettivo di questa fase

Dopo aver concluso la fase di object detection e aver scelto come modello di riferimento **YOLOv11 exp11b**, è stata avviata la seconda parte del progetto, orientata alla ricostruzione della topologia del diagramma elettrico.

L'idea è costruire una pipeline incrementale che, a partire dalle detection del modello, consenta di arrivare progressivamente a:

1. identificazione univoca dei componenti presenti nel diagramma;
2. stima dei terminali dei componenti semplici;
3. estrazione dei fili;
4. ricostruzione delle net;
5. associazione terminale-net;
6. costruzione di una struttura a grafo.

In questa fase iniziale si è lavorato su una **singola immagine pilota**, in modo da validare ogni blocco della pipeline prima di estendere il processo a più immagini.

1. Analisi preliminare delle classi del dataset

1.1 Costruzione dei file di riepilogo classi

Per capire come fosse distribuito il dataset ed individuare le classi più adatte per l'MVP iniziale, è stato creato uno script dedicato al riepilogo delle classi.

File prodotti

- metadata/class_summary_global.csv
- metadata/class_summary_by_split.csv

1.2 Scopo dei due file

class_summary_global.csv

Contiene, per ogni classe:

- class_id
- class_name
- total_count

Serve a capire quante istanze totali di ciascuna classe sono presenti nell'intero dataset.

class_summary_by_split.csv

Contiene, per ogni classe:

- class_id
- class_name
- train_count
- valid_count
- test_count
- total_count

Serve a capire come le classi sono distribuite nei tre split del dataset.

1.3 Utilità pratica

Questa analisi è stata utile per:

- verificare il mapping tra ID numerico e nome classe;
- capire quali classi fossero sufficientemente rappresentate;
- selezionare un primo sottoinsieme di classi su cui lavorare nella pipeline topologica.

2. Definizione del file `class_terminals.yaml`

2.1 Obiettivo

È stato creato il file:

- `metadata/class_terminals.yaml`

Questo file rappresenta il primo livello di conoscenza strutturata sulle classi del dataset e definisce, per ogni classe selezionata:

- nome della classe;
- tipo simbolico;
- uso o meno nella stima dei terminali;
- uso o meno nel mascheramento del simbolo;
- posizione relativa dei terminali rispetto al bounding box.

2.2 Motivazione

Lo scopo del file non è solo descrivere i componenti, ma separare già da subito due casi diversi:

1. **classi usate per la stima dei terminali**
2. **classi usate solo per il mascheramento del simbolo**

Questo è particolarmente importante per componenti come il **Mosfet**, che nella prima fase non viene ancora trattato topologicamente, ma deve comunque essere rilevato e mascherato per non disturbare la successiva estrazione dei fili.

2.3 Versione attuale del file

Le classi selezionate per l'immagine pilota sono:

- 31 → Voltage_Source
- 22 → Resistor
- 4 → Capacitor
- 26 → Terminal
- 9 → GND
- 16 → Mosfet

Il Mosfet è attualmente configurato con:

- `use_for_terminals: false`
- `use_for_masking: true`

mentre le altre classi pilota sono configurate per essere usate sia per la stima dei terminali sia per il mascheramento.

3. Organizzazione degli script per la seconda parte del progetto

Per separare chiaramente la nuova pipeline dalla parte precedente legata ad augmentation, risultati e utility generiche, è stata creata una nuova cartella dedicata:

- `scripts/topology/`

4. Script 01_detect_components.py

4.1 Obiettivo

Il primo script della nuova pipeline è:

```
scripts/topology/01_detect_components.py
```

Questo script ha il compito di:

- caricare il modello `best.pt` di YOLOv11 exp11b;
- leggere il file `metadata/class_terminals.yaml` ;
- selezionare solo le classi definite nel file yaml;
- eseguire la detection sulle immagini in input;
- salvare un file JSON strutturato;
- salvare anche un'immagine di debug con i bounding box disegnati.

4.2 Input

Per la prima prova è stata scelta una singola immagine pilota:

`data/pilot_images/pilot_001.jpg`

4.3 Output prodotti

JSON detection

`outputs/topology/01_detect_components/pilot_001.json`

Immagine debug

`outputs/topology/01_detect_components/debug_images/pilot_001_detect.jpg`

4.4 Contenuto del JSON

Il file JSON contiene:

- informazioni sull'immagine;
- classi abilitate per detection, terminali e masking;
- lista dei componenti rilevati, ciascuno con:
 - `class_id`
 - `class_name`
 - `conf`
 - `bbox`
 - `symbol_type`
 - `use_for_terminals`
 - `use_for_masking`

4.5 Risultato ottenuto su pilot_001

La detection sull'immagine pilota ha prodotto **13 componenti**:

- 1 Terminal
- 3 Capacitor
- 3 Resistor
- 1 Mosfet
- 4 GND
- 1 Voltage_Source

Questo risultato è coerente con il contenuto visivo dell'immagine pilota e può essere considerato soddisfacente come base per la fase successiva.

4.6 Immagine di debug

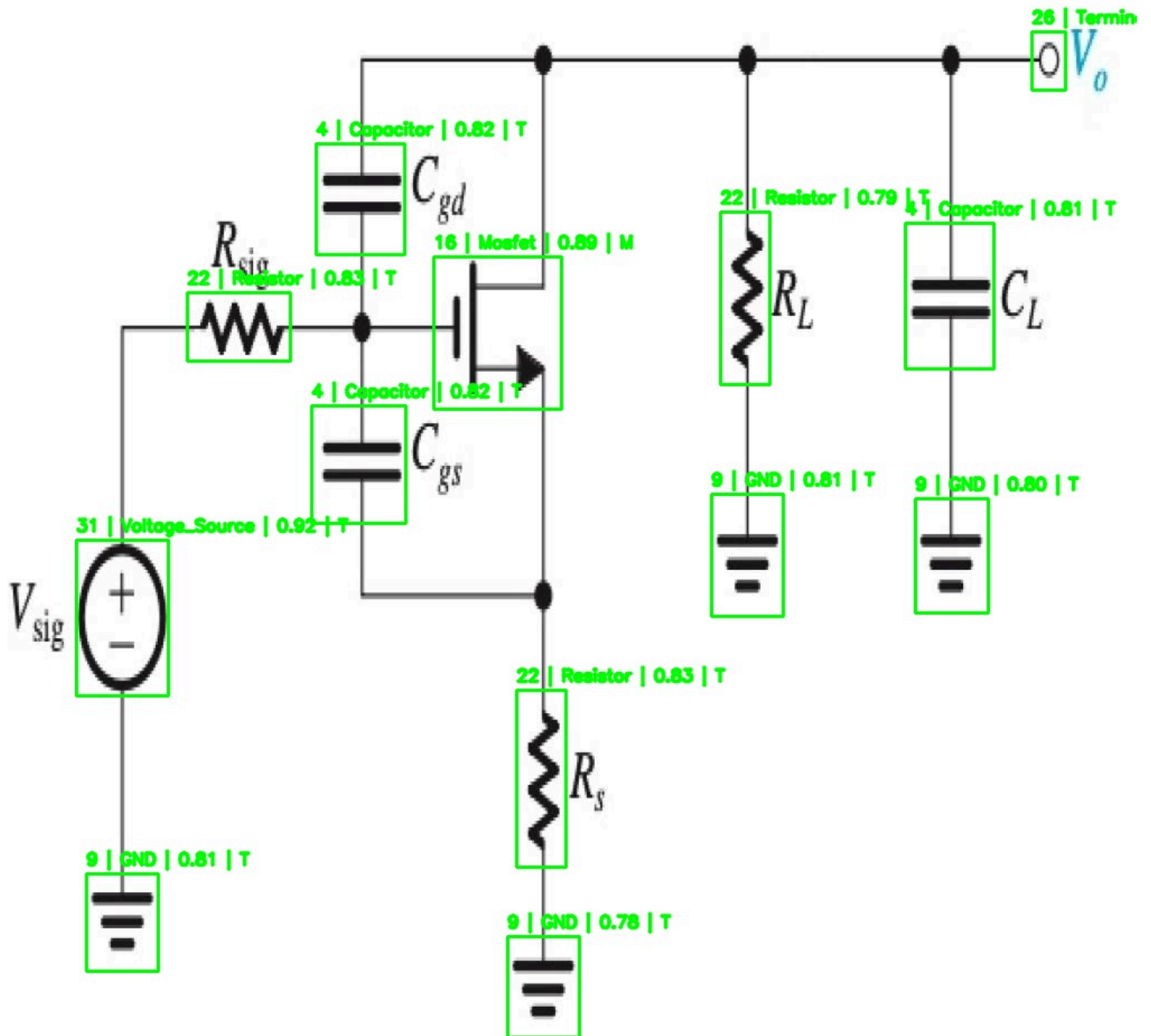


Figura 1. Risultato della detection iniziale su pilot_001.jpg tramite YOLOv11 exp11b. I bounding box mostrano i componenti rilevati appartenenti alle classi definite in class_terminals.yaml. In questa fase vengono mantenuti anche i componenti usati solo per masking, come il Mosfet.

5. Script 02_assign_instances.py

5.1 Obiettivo

Il secondo script della pipeline è:

scripts/topology/02_assign_instances.py

Questo script ha il compito di:

- leggere i file JSON prodotti da 01_detect_components.py;
- raggruppare i componenti per class_id;
- ordinare i componenti secondo una regola geometrica;
- assegnare un instance_id univoco del tipo:
 - 22.1
 - 22.2
 - 4.1
 - 9.1
 - ecc.;
- salvare il nuovo JSON aggiornato;
- salvare anche una nuova immagine debug con le istanze annotate.

5.2 Convenzione scelta per la numerazione

Per la numerazione delle istanze è stata scelta la convenzione:

`SORT_ORDER = "xy"`

cioè: da sinistra verso destra e a parità di x, dall'alto verso il basso.

Questa convenzione è stata preferita perché più naturale rispetto alla lettura tipica dei circuiti, che spesso si sviluppano lungo una direzione sinistra-destra.

L'instance_id non ha ancora significato elettrico o topologico: serve unicamente a fornire un identificatore univoco, stabile e leggibile all'interno del diagramma.

5.3 Output prodotti

JSON con istanze

`outputs/topology/02_assign_instances/pilot_001.json`

Immagine debug con istanze

`outputs/topology/02_assign_instances/debug_images/pilot_001_instances.jpg`

5.4 Risultato ottenuto su pilot_001

Dopo l'assegnazione delle istanze, i componenti risultano etichettati come segue:

- 26.1 → Terminal
- 4.1, 4.2, 4.3 → Capacitor
- 22.1, 22.2, 22.3 → Resistor
- 16.1 → Mosfet
- 9.1, 9.2, 9.3, 9.4 → GND
- 31.1 → Voltage_Source

Il risultato è coerente con il numero di componenti rilevati e costituisce la base per la successiva stima dei terminali.

5.5 Immagine di debug

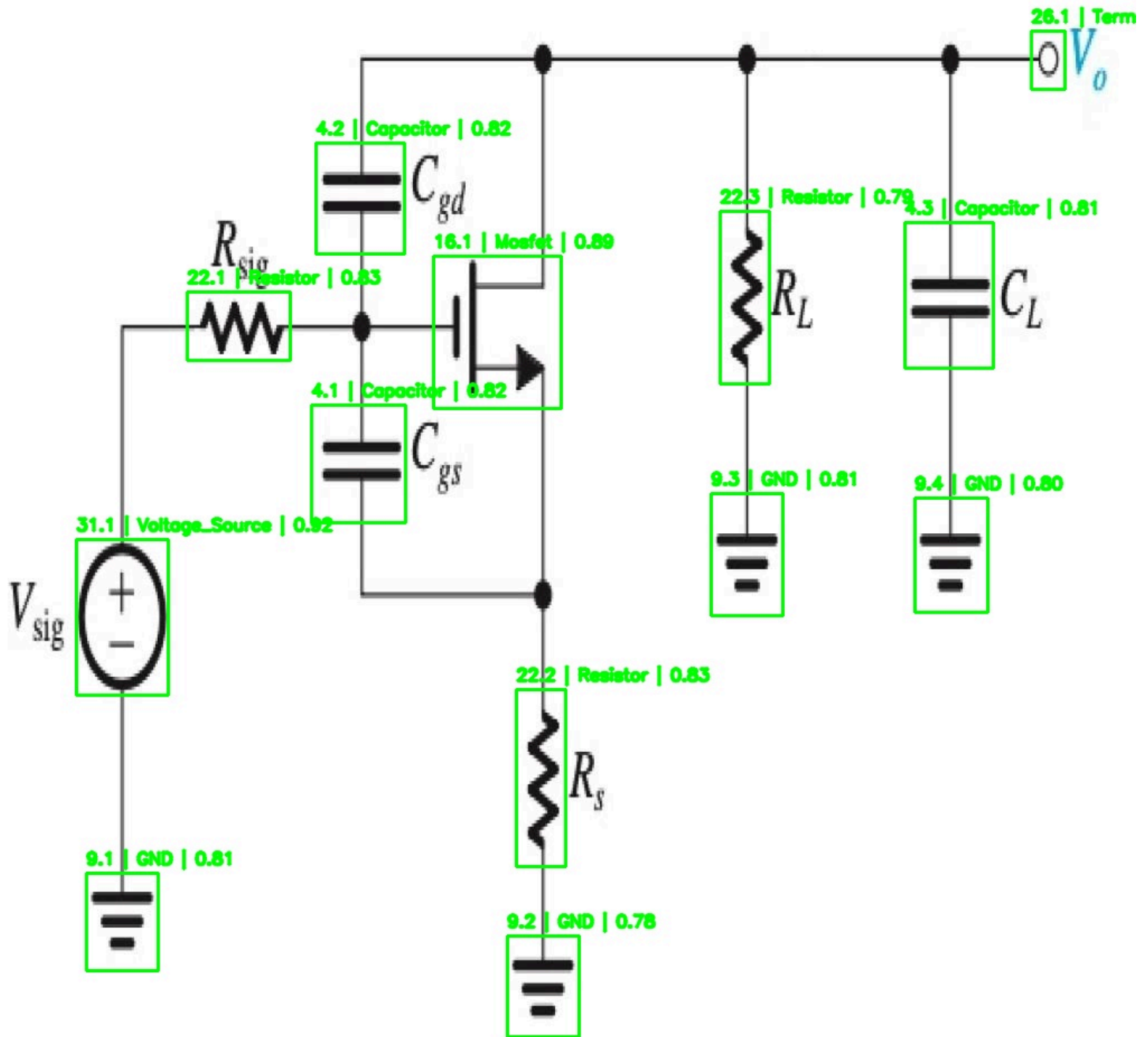


Figura 2. Risultato dello script 02_assign_instances.py su pilot_001.jpg. Ogni componente rilevato è stato associato a un identificatore univoco (instance_id) ottenuto raggruppando per classe e ordinando gli oggetti secondo la convenzione xy.

6. Script `03_estimate_terminals.py`

6.1 Obiettivo

Il terzo script della pipeline è:

```
scripts/topology/03_estimate_terminals.py
```

Questo script ha il compito di:

- leggere i file JSON prodotti da `02_assign_instances.py` ;
- leggere il file `metadata/class_terminals.yaml` ;
- stimare, per ogni componente con `use_for_terminals: true` , le coordinate dei terminali nell'immagine;
- salvare tali terminali sia all'interno del singolo componente sia in una lista globale;
- salvare una nuova immagine di debug con terminali e identificativi associati.

6.2 Evoluzione della logica di stima

Nella prima versione dello script, i terminali venivano calcolati usando direttamente le posizioni specificate nel file yaml, ad esempio `left/right` oppure `top/bottom` .

Questa prima soluzione si è rivelata insufficiente per componenti come:

- Capacitor
- Resistor

perché nell'immagine pilota essi possono comparire sia in orientazione orizzontale sia in orientazione verticale.

Per questo motivo la logica è stata migliorata introducendo una stima automatica dell'orientazione del componente basata sul **rapporto tra altezza e larghezza del bounding box**.

In particolare:

- se il bounding box è più alto che largo, il componente viene trattato come **verticale**;
- se il bounding box è più largo che alto, il componente viene trattato come **orizzontale**.

Questo consente, per i componenti a due terminali, di scegliere automaticamente tra:

- `left/right`
- `top/bottom`

6.3 Aggiornamento di `class_terminals.yaml`

Per rendere possibile questa logica, il file `class_terminals.yaml` è stato esteso introducendo, per alcune classi, un campo di strategia terminali.

In particolare:

- `Voltage_Source` mantiene una strategia fissa (`top/bottom`);
- `GND` mantiene una strategia fissa (`top`);
- `Terminal` mantiene una strategia fissa (`left`);
- `Resistor` e `Capacitor` utilizzano invece una strategia automatica basata sull'aspect ratio del bounding box.

Questo aggiornamento ha permesso di correggere la stima dei terminali, in particolare sui condensatori verticali e sui resistori verticali dell'immagine pilota.

6.4 Output prodotti

JSON con terminali stimati

`outputs/topology/03_estimate_terminals/pilot_001.json`

Immagine debug con terminali

`outputs/topology/03_estimate_terminals/debug_images/pilot_001_terminals.jpg`

6.5 Contenuto del JSON

Nel nuovo file JSON, ogni componente contiene anche:

- la lista dei terminali stimati;
- l'eventuale orientazione stimata (`horizontal` o `vertical`) per i componenti con orientazione automatica.

Inoltre, a livello globale viene aggiunta una lista:

- `terminals`

contenente tutti i terminali dell'immagine.

Ogni terminale è descritto da campi come:

- `terminal_id`
- `instance_id`
- `component_class_id`
- `component_class_name`
- `name`
- `relative_position`
- `estimated_orientation`
- `x`
- `y`

6.6 Risultato ottenuto su `pilot_001`

Dopo la correzione della logica di orientazione, i terminali risultano coerenti con la geometria reale del diagramma.

In particolare:

- i condensatori verticali vengono trattati con terminali `top/bottom` ;
- i resistori verticali vengono trattati con terminali `top/bottom` ;
- il resistore orizzontale `22.1` mantiene correttamente i terminali `left/right` ;
- la `Voltage_Source` mantiene i terminali `top/bottom` ;
- i simboli `GND` mantengono il terminale `top` ;
- il `Terminal` di uscita mantiene il terminale sul lato sinistro;
- il `Mosfet` non riceve terminali, poiché è attualmente usato solo per masking.

Nel caso dell'immagine pilota sono stati stimati complessivamente **19 terminali**.

6.7 Immagine di debug

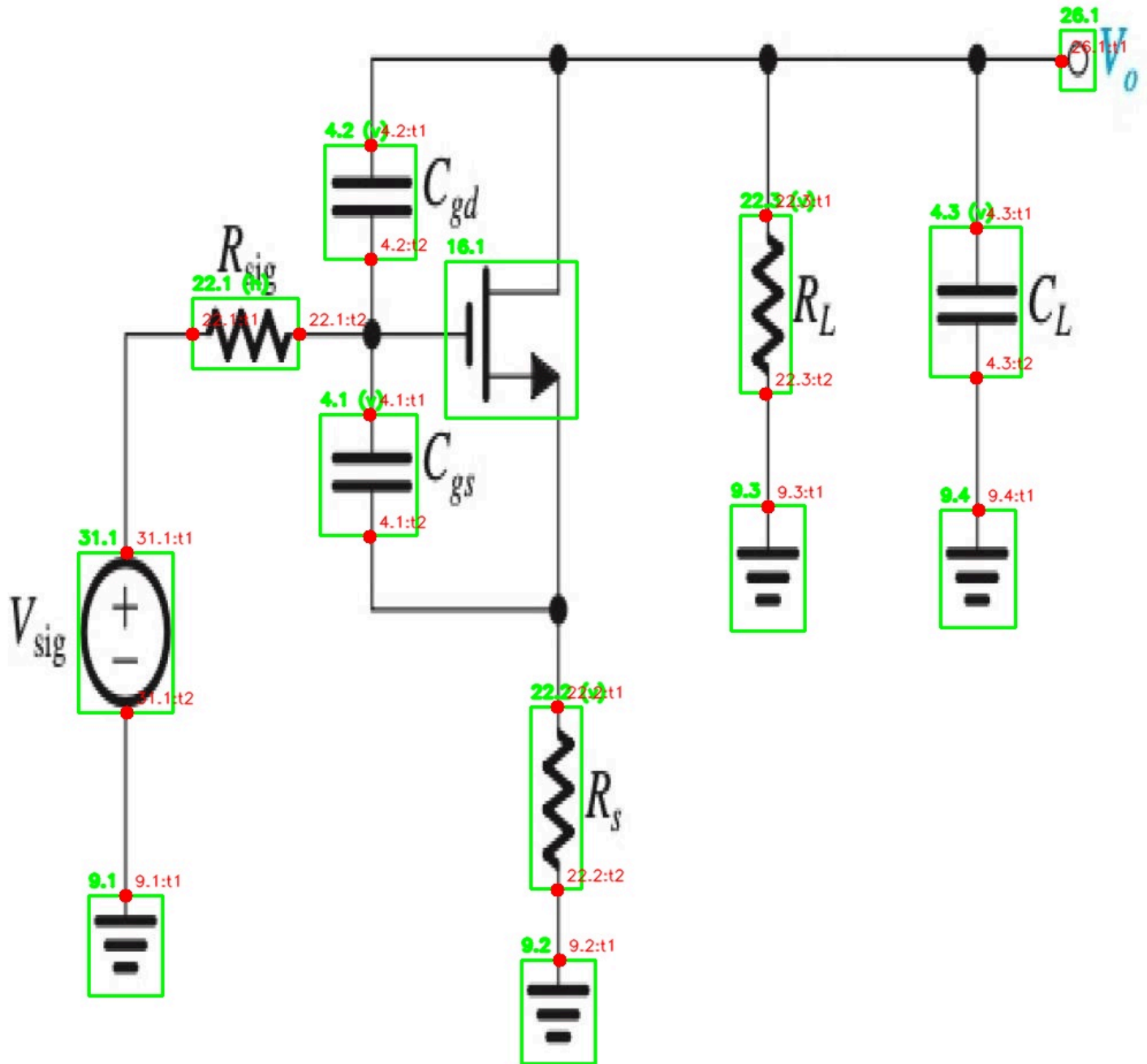


Figura 3. Risultato dello script `03_estimate_terminals.py` su `pilot_001.jpg`. I terminali stimati sono mostrati come punti rossi e associati ai rispettivi `terminal_id`. La stima dell'orientazione per resistori e condensatori è stata corretta introducendo una logica automatica basata sulle proporzioni del bounding box.

7. Script 04_extract_wires.py

7.1 Obiettivo

Il quarto script della pipeline è:

```
scripts/topology/04_extract_wires.py
```

Questo script ha il compito di:

- leggere il file JSON prodotto da 03_estimate_terminals.py ;
- leggere l'immagine originale;
- mascherare i componenti marcati con `use_for_masking: true` ;
- preservare un piccolo intorno dei terminali stimati;
- produrre una versione semplificata dell'immagine in cui i simboli siano rimossi ma i fili restino visibili;
- applicare binarizzazione, closing e skeletonization;
- salvare diversi output intermedi per il debug.

7.2 Logica generale

L'idea di questo blocco è separare, per quanto possibile, il **layer dei collegamenti** dal layer dei simboli.

La procedura seguita è la seguente:

1. conversione dell'immagine originale in scala di grigi;
2. costruzione di una maschera dei componenti da rimuovere;
3. apertura di piccoli varchi in corrispondenza dei terminali stimati, per non cancellare completamente i punti di attacco dei fili;
4. applicazione della maschera all'immagine;
5. binarizzazione inversa per ottenere fili e testo come foreground;
6. applicazione di un'operazione di **closing**;
7. applicazione di un filtro opzionale per rimuovere piccoli componenti connessi;
8. applicazione della **skeletonization** per ridurre i fili a una traccia sottile.

7.3 Migliorie introdotte

Rispetto alla prima versione del blocco 04, sono state introdotte due migliorie principali:

1. Salvataggio della maschera pura

Oltre all'overlay `mask_debug`, viene salvata anche la maschera binaria vera e propria:

- `component_mask.png`

Questo rende più semplice controllare esattamente quali aree dell'immagine vengono coperte.

2. Filtro opzionale per piccoli componenti connessi

È stato aggiunto un filtro per connected components di piccola area, con i parametri:

- `ENABLE_SMALL_COMPONENT_FILTER = True`
- `MIN_COMPONENT_AREA = 40`

Questo filtro non elimina completamente il testo, ma può rimuovere alcuni piccoli residui non utili prima della skeletonization.

7.4 Limiti noti della versione attuale

In questa fase il testo **non viene ancora rimosso esplicitamente**.

Di conseguenza, nelle immagini:

- `binary`
- `closed`
- `filtered`
- `skeleton`

possono ancora comparire residui testuali come:

Questa limitazione è attualmente accettata, poiché l'obiettivo del pilot è verificare la correttezza della logica di base per l'estrazione dei collegamenti.

7.5 Parametri utilizzati

Nella versione attuale sono stati usati i seguenti parametri:

- `mask_shrink_factor = 0.88`
- `terminal_keep_radius = 10`
- `closing_kernel_size = 3`
- `closing_iterations = 1`
- `small_component_filter.enabled = true`
- `small_component_filter.min_component_area = 40`

7.6 Output prodotti

Lo script produce e salva:

Overlay maschera

`outputs/topology/04_extract_wires/mask_debug/pilot_001_mask_debug.jpg`

Maschera pura

`outputs/topology/04_extract_wires/component_mask/pilot_001_component_mask.png`

Immagine grayscale mascherata

`outputs/topology/04_extract_wires/masked_gray/pilot_001_masked_gray.png`

Immagine binaria

`outputs/topology/04_extract_wires/binary/pilot_001_binary.png`

Immagine dopo closing

`outputs/topology/04_extract_wires/closed/pilot_001_closed.png`

Immagine dopo filtro piccoli componenti

`outputs/topology/04_extract_wires/filtered/pilot_001_filtered.png`

Skeleton finale

`outputs/topology/04_extract_wires/skeleton/pilot_001_skeleton.png`

Inoltre, il file JSON prodotto viene aggiornato con una nuova sezione:

- `wire_extraction`

contenente:

- i parametri usati;
- le informazioni sul filtro;
- i path di tutti gli output intermedi.

7.7 Valutazione qualitativa su `pilot_001`

Nel caso dell'immagine pilota, il risultato del blocco 04 è stato ritenuto soddisfacente per una prima versione.

In particolare:

- la maschera dei componenti è coerente con i bounding box e con i terminali stimati;
- i simboli vengono rimossi in modo consistente;
- i fili principali del circuito restano visibili;
- i piccoli varchi attorno ai terminali vengono preservati;
- la topologia generale del circuito è ancora leggibile dopo skeletonization.

Il problema principale che resta aperto è la presenza di testo residuo nelle immagini binarie e nello skeleton.

7.8 Immagini di debug

7.8.1 Overlay della maschera

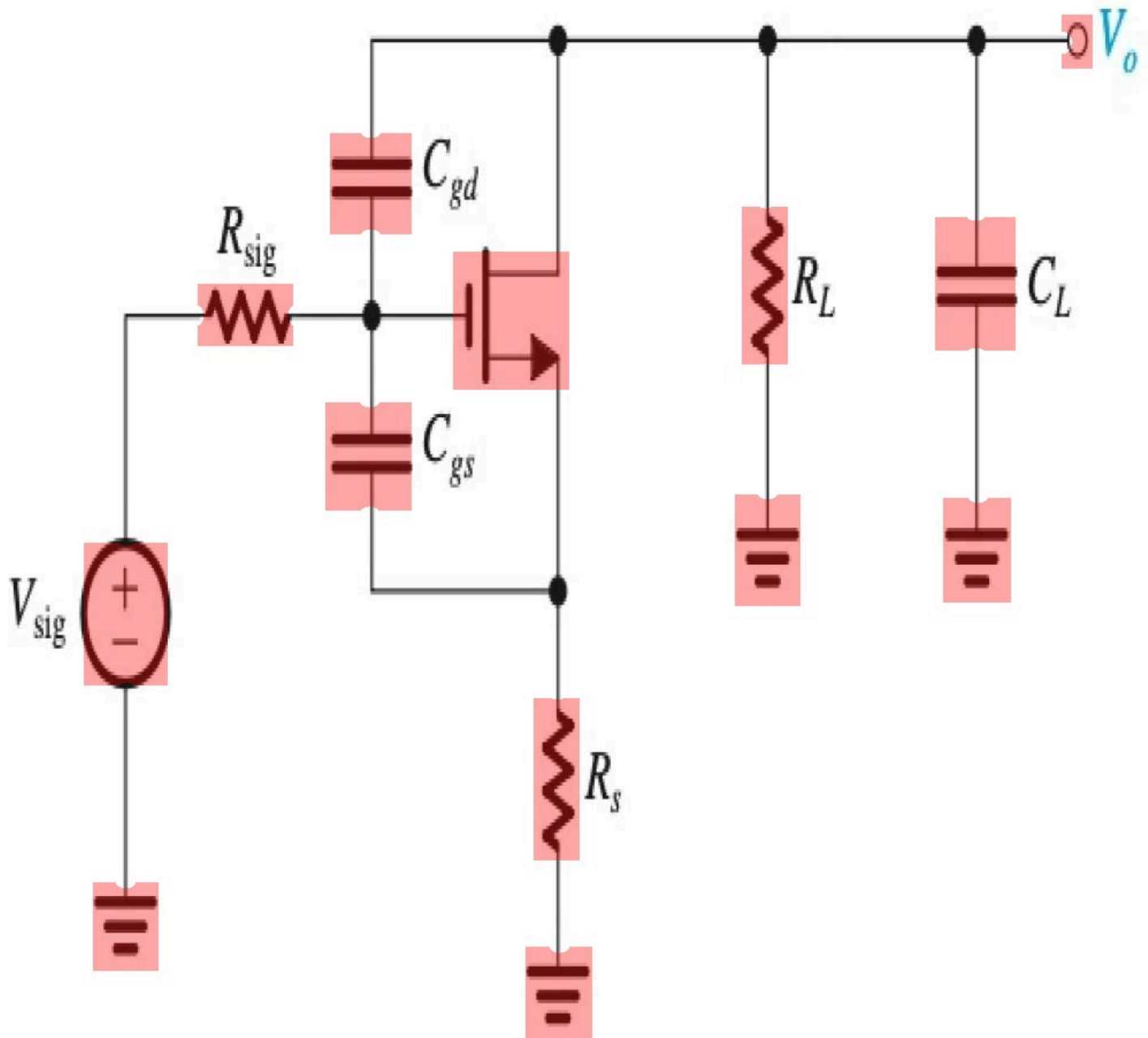


Figura 4. Overlay della maschera applicata ai componenti. Le aree in rosso indicano i simboli che verranno rimossi nella fase di estrazione dei fili. I piccoli varchi in corrispondenza dei terminali permettono di preservare i punti di contatto con i collegamenti.

7.8.2 Maschera pura dei componenti

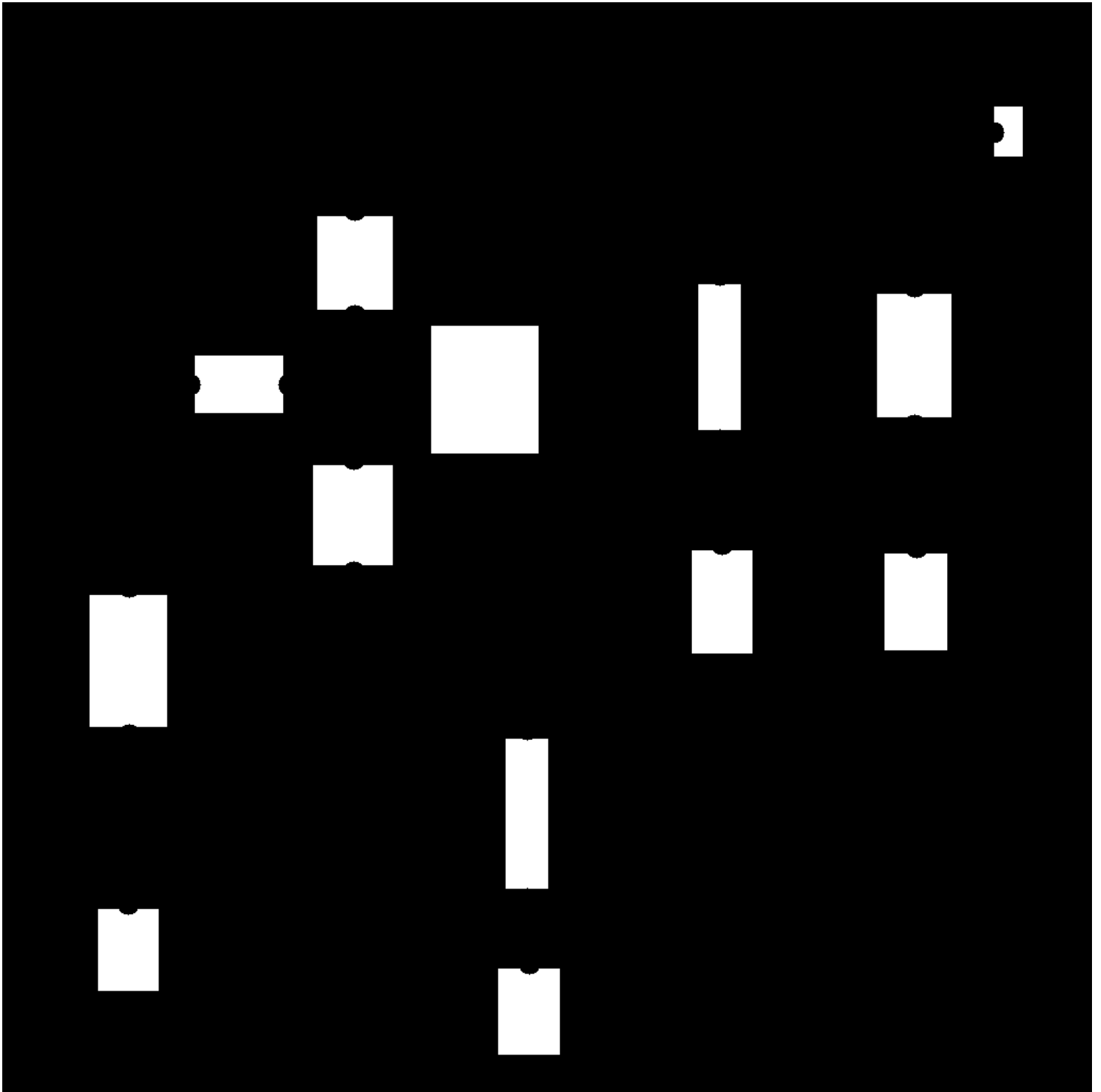


Figura 5. Maschera binaria pura dei componenti. Le aree bianche corrispondono ai simboli da mascherare, mentre i piccoli intagli neri evidenziano i punti preservati in corrispondenza dei terminali stimati.

7.8.3 Immagine in scala di grigi mascherata

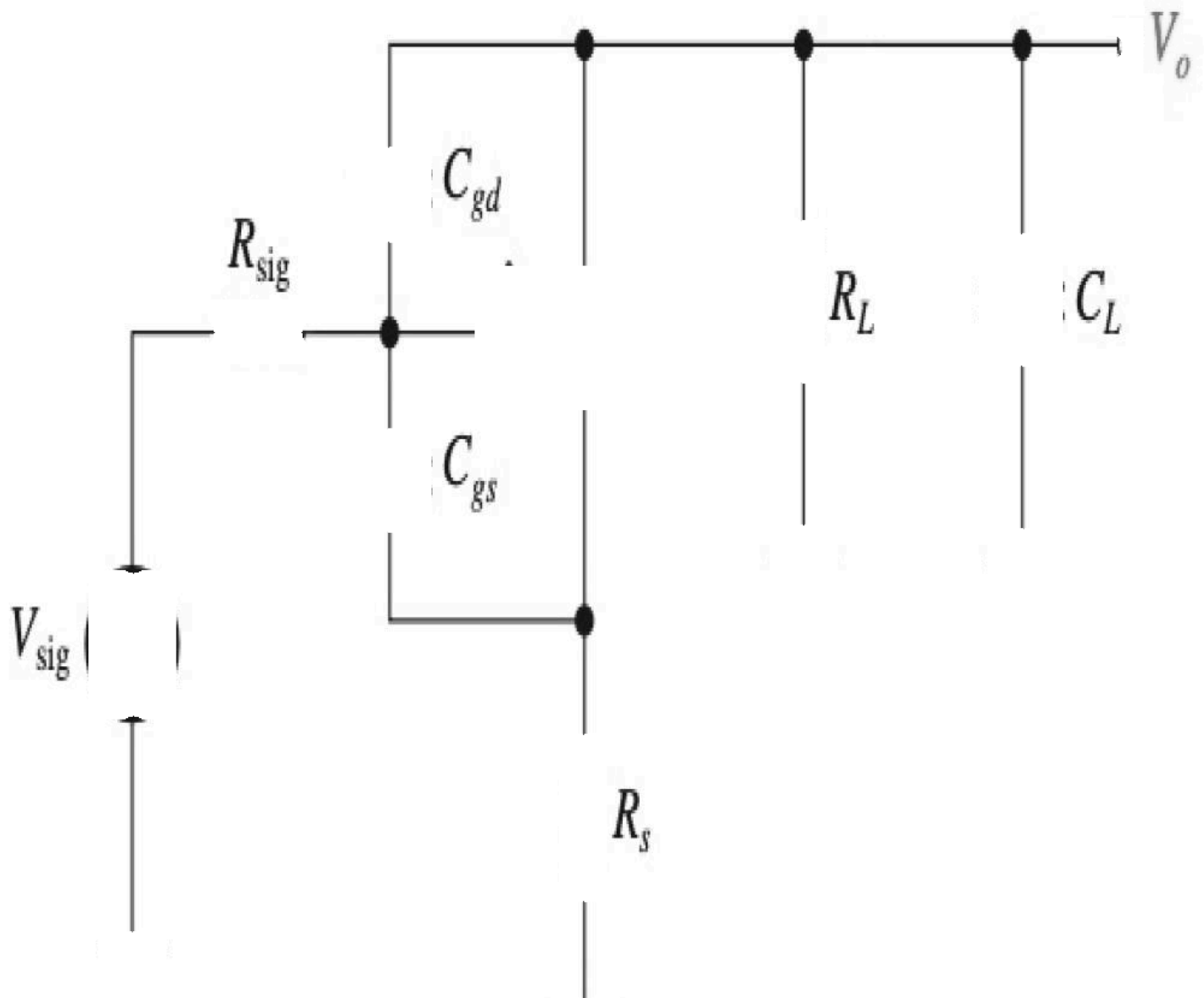


Figura 6. Immagine in scala di grigi dopo l'applicazione della maschera. I simboli dei componenti sono stati rimossi, mentre i fili principali del circuito sono rimasti visibili.

7.8.4 Immagine binaria

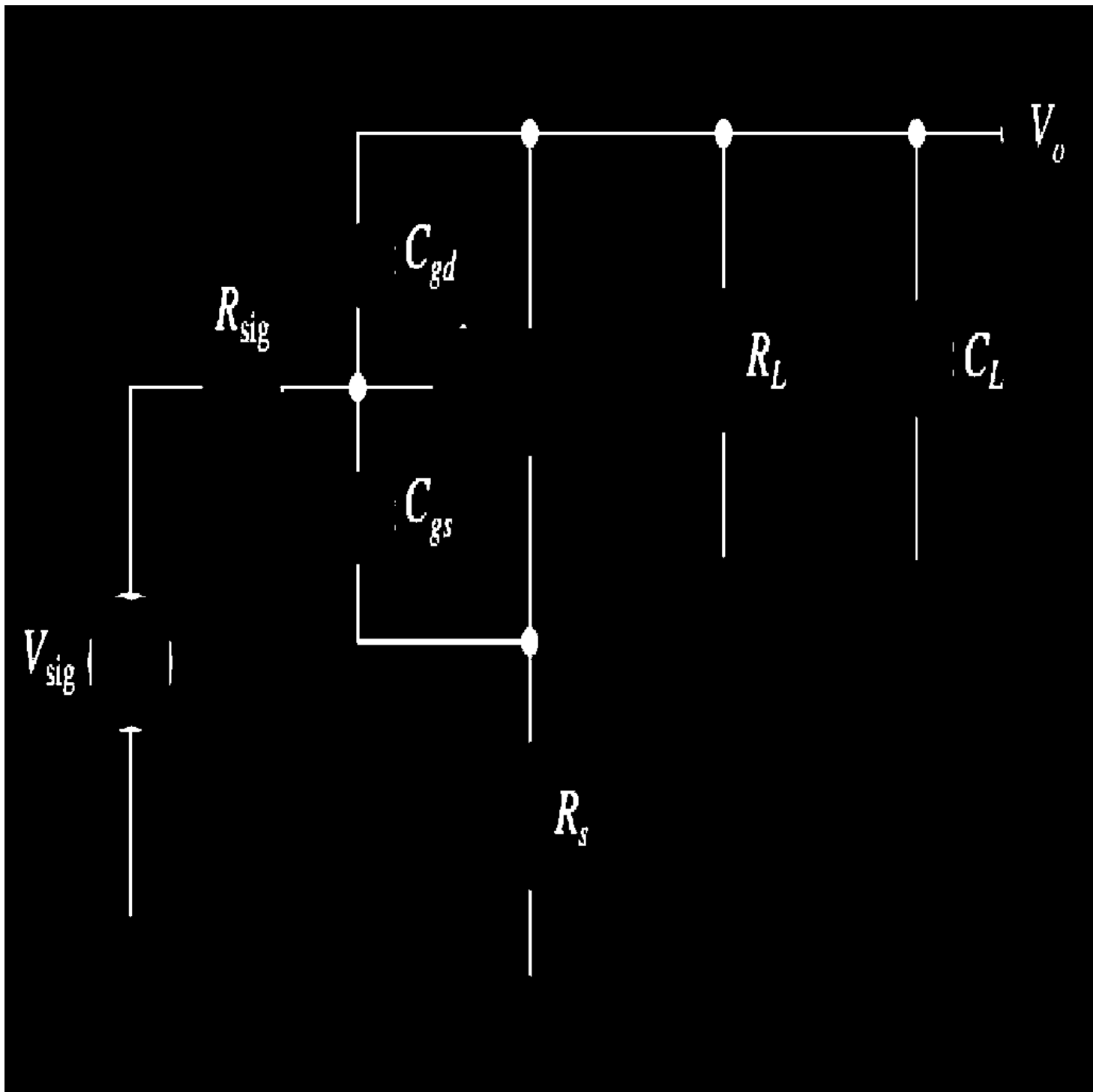


Figura 7. Immagine binaria ottenuta tramite threshold inverso. I fili e parte del testo vengono trattati come foreground bianco su sfondo nero.

7.8.5 Immagine dopo closing

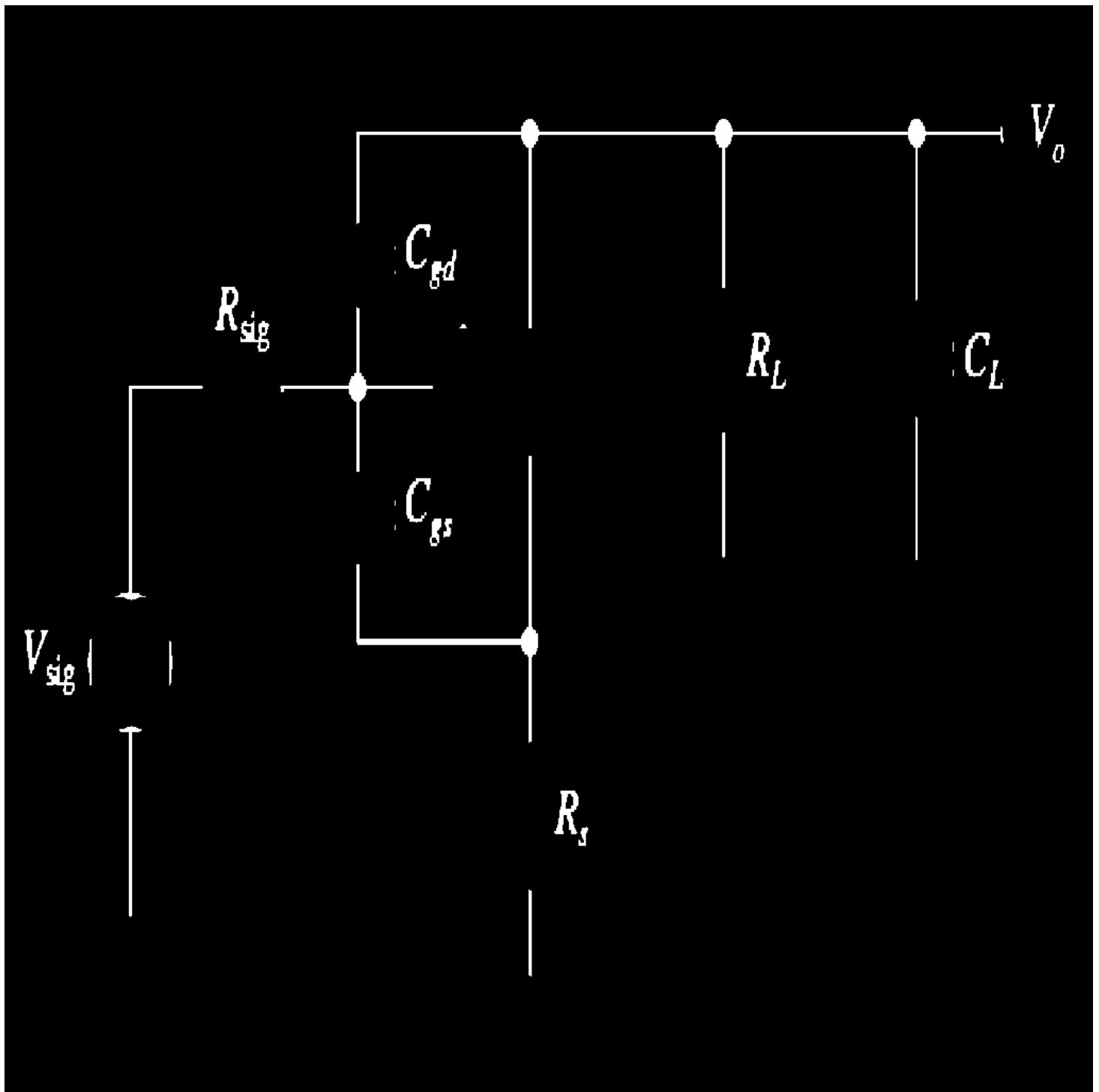


Figura 8. Risultato dell'operazione di closing. Nel caso dell'immagine pilota l'effetto è contenuto, poiché la continuità dei fili era già sufficientemente buona nella binaria iniziale.

7.8.6 Immagine dopo filtro dei piccoli componenti

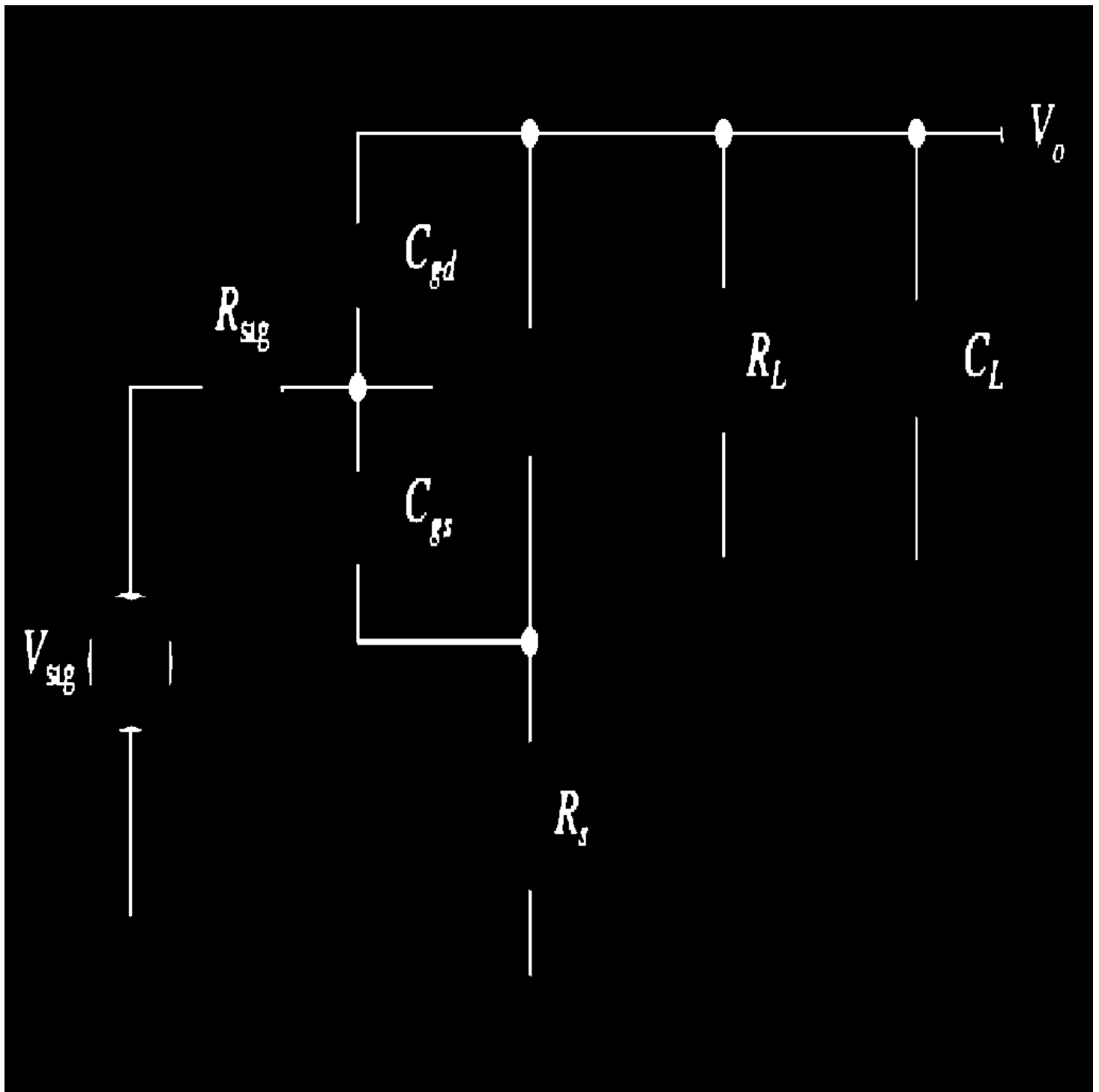


Figura 9. Immagine ottenuta dopo la rimozione dei piccoli componenti connessi. Questa fase riduce parte del rumore, pur non eliminando completamente il testo presente nello schema.

7.8.7 Skeleton finale

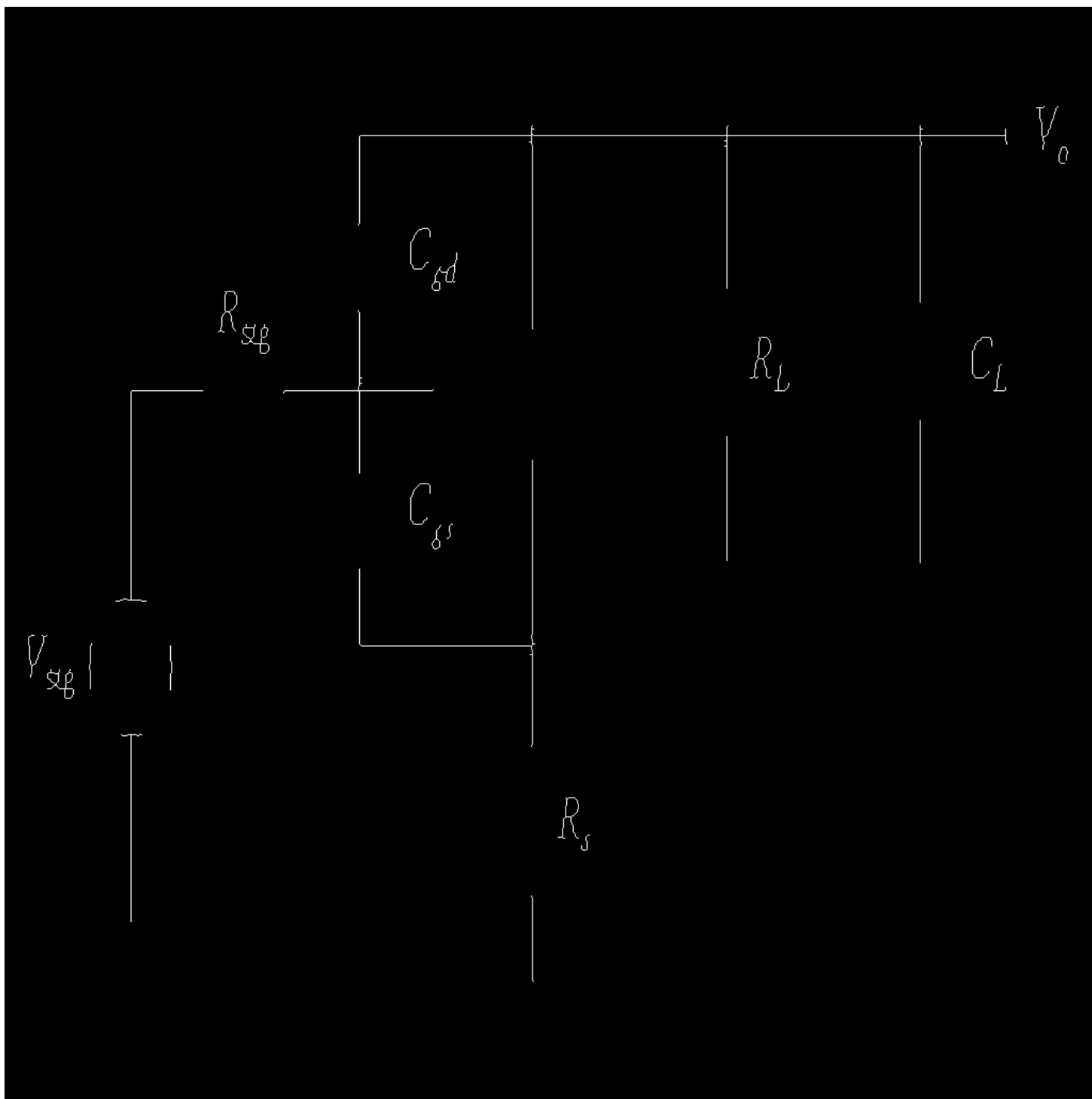


Figura 10. Skeleton finale ottenuto dalla versione filtrata dell'immagine binaria. I fili risultano ridotti a una traccia sottile, utile per la fase successiva di identificazione delle net.

8. Script 05_build_nets.py

8.1 Obiettivo

Il quinto script della pipeline è:

```
scripts/topology/05_build_nets.py
```

Questo script ha il compito di:

- leggere il file JSON prodotto da 04_extract_wires.py ;
- leggere lo skeleton dei fili generato nel blocco precedente;
- individuare le connected components dello skeleton;
- considerare ciascuna connected component come una possibile net;
- verificare quali terminali cadono in prossimità di ciascuna connected component;
- filtrare le componenti che non hanno sufficiente consistenza geometrica o topologica;
- assegnare identificativi del tipo N1 , N2 , N3 , ...;
- salvare una rappresentazione strutturata delle net trovate;
- produrre immagini di debug utili per la validazione visiva.

8.2 Logica generale

L'idea di questo blocco è trasformare lo skeleton dei fili in una prima rappresentazione esplicita delle reti elettriche.

La procedura seguita è la seguente:

1. caricamento dell'immagine skeleton prodotta dal blocco 04;
2. identificazione delle connected components tramite `connectedComponentsWithStats` ;
3. costruzione di una lista di candidate net;
4. ricerca, per ogni terminale stimato, delle connected components presenti in una piccola finestra attorno al terminale;
5. associazione di ogni terminale alle componenti candidate vicine;
6. filtraggio delle componenti troppo piccole o prive di terminali rilevanti;
7. ri-etichettatura delle componenti mantenute come net finali (N1 , N2 , N3 , ...);
8. salvataggio di una mappa label numerica e di immagini di visualizzazione.

8.3 Motivazione della strategia adottata

In questa fase si è scelto di considerare come net valide solo le connected components che:

- hanno una dimensione minima sufficiente;
- risultano vicine ad almeno un terminale stimato.

Questa scelta è stata fatta perché nello skeleton sono ancora presenti residui di testo e piccoli frammenti di rumore.

Il vincolo “la net deve toccare almeno un terminale” consente di evitare che molte componenti spurie vengano interpretate come vere reti del circuito.

8.4 Parametri utilizzati

Nella versione attuale sono stati usati i seguenti parametri:

- `terminal_net_radius = 8`
- `min_net_pixels = 8`
- `min_connected_terminals = 1`
- `net_sort_order = "xy"`

L'ordinamento `xy` è stato mantenuto per coerenza con la numerazione già adottata per le istanze dei componenti.

8.5 Output prodotti

Lo script produce e salva:

JSON con net costruite

`outputs/topology/05_build_nets/pilot_001.json`

Mappa label delle net

`outputs/topology/05_build_nets/label_maps/pilot_001_net_labels.npy`

Immagine `net_map`

`outputs/topology/05_build_nets/net_map/pilot_001_net_map.png`

Immagine overlay

outputs/topology/05_build_nets/overlay/pilot_001_net_overlay.jpg

Inoltre, il file JSON viene aggiornato con:

- una lista `nets` ;
- il numero totale di net trovate (`n_nets`);
- una sezione `net_building` con i parametri usati e con informazioni di debug aggiuntive.

8.6 Contenuto del JSON

Per ogni net vengono salvati campi del tipo:

- `net_id`
- `net_index`
- `source_label`
- `pixel_count`
- `bbox`
- `connected_terminal_ids`
- `n_connected_terminals`

Questo consente di sapere, per ogni net individuata:

- da quale connected component dello skeleton deriva;
- quanti pixel la compongono;
- quali terminali risultano collegati o vicini a essa.

8.7 Risultato ottenuto su `pilot_001`

Nel caso dell'immagine pilota, lo script ha trovato:

- 27 connected components candidate;
- 8 net mantenute;
- 19 componenti candidate scartate.

Le net finali individuate sono:

- N1
- N2

- N3
- N4
- N5
- N6
- N7
- N8

8.8 Interpretazione qualitativa delle net trovate

Le otto net individuate risultano coerenti con la struttura del diagramma pilota.

In particolare:

- N1 corrisponde al ramo inferiore della sorgente, collegato al simbolo di massa;
- N2 rappresenta il collegamento tra la sorgente e il resistore di ingresso;
- N3 rappresenta il nodo centrale sinistro, in cui convergono il resistore di ingresso e i due condensatori di gate;
- N4 rappresenta il nodo inferiore centrale, collegato al condensatore inferiore e al resistore R_s ;
- N5 rappresenta il ramo tra R_s e il relativo simbolo di massa;
- N6 rappresenta la barra superiore comune, che collega il condensatore superiore, il resistore R_L , il condensatore C_L e il terminale di uscita;
- N7 rappresenta il ramo verticale di R_L verso massa;
- N8 rappresenta il ramo verticale di C_L verso massa.

Dal punto di vista grafico, il risultato è plausibile e coerente con l'attuale livello di astrazione della pipeline.

8.9 Osservazioni importanti

1. Il numero di net può sembrare elevato, ma è coerente

Le net trovate non rappresentano ancora una netlist elettrica “semantica” completa, ma una segmentazione geometrica dei tratti di filo connessi nello skeleton.

2. Il Mosfet spezza intenzionalmente la continuità

Poiché il Mosfet è ancora trattato come:

- `use_for_masking: true`

- `use_for_terminals: false`

esso interrompe la continuità del circuito in quella zona.

Questo significa che, in questa fase, la pipeline non tenta ancora di ricostruire le connessioni che passano attraverso i terminali del Mosfet .

3. I simboli GND non vengono ancora fusi semanticamente

Attualmente i diversi rami che terminano con simboli di massa vengono trattati come net separate se risultano separate nello skeleton.

In una fase successiva sarà possibile introdurre una regola semantica per unificare le masse.

8.10 Immagini di debug

8.10.1 Mappa delle net

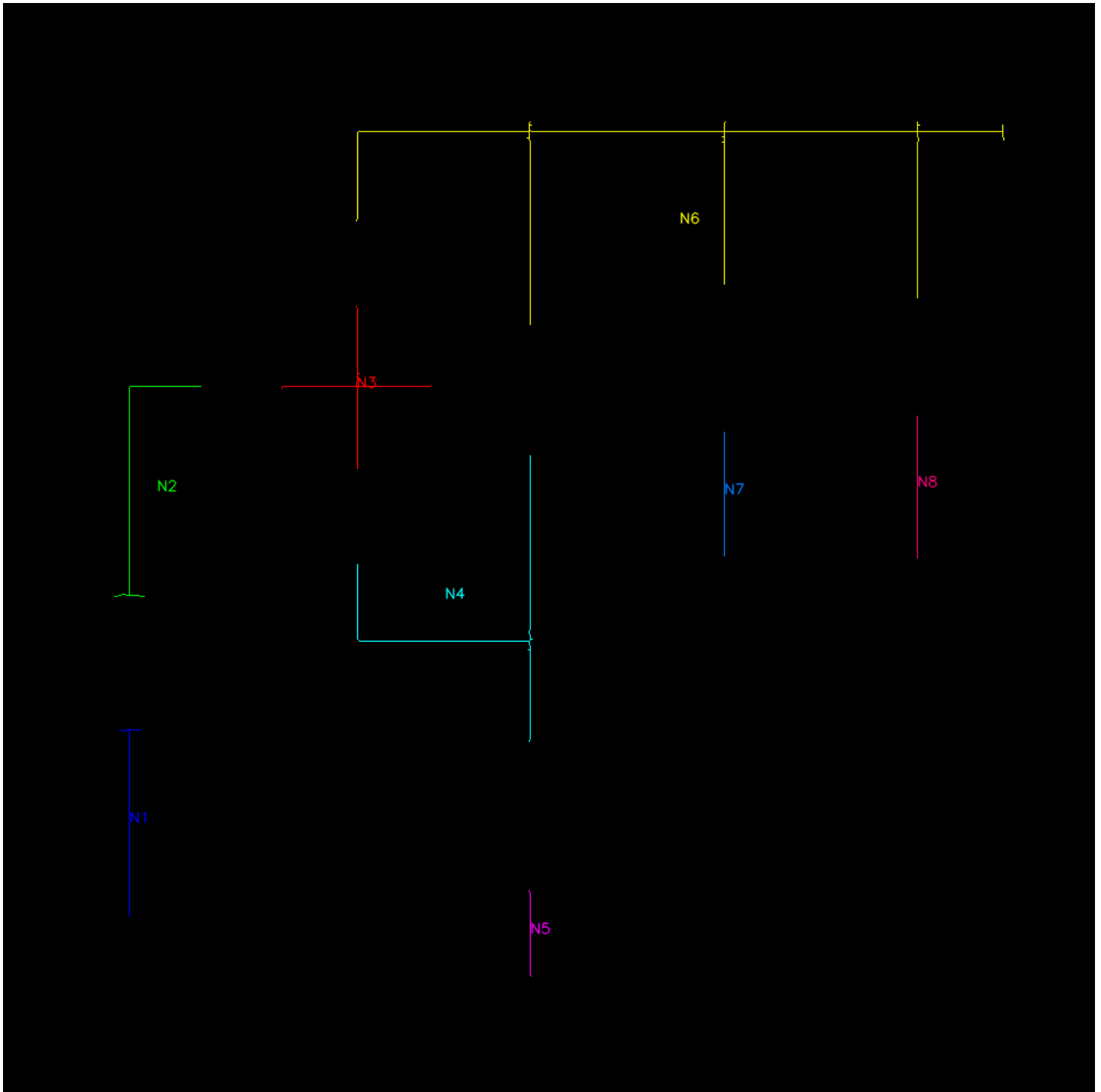


Figura 11. Risultato dello script `05_build_nets.py` sotto forma di mappa delle net. Ogni connected component mantenuta è stata convertita in una net e rappresentata con un colore distinto e con un identificativo del tipo N1 , N2 , N3 ,

8.10.2 Overlay delle net sull'immagine originale

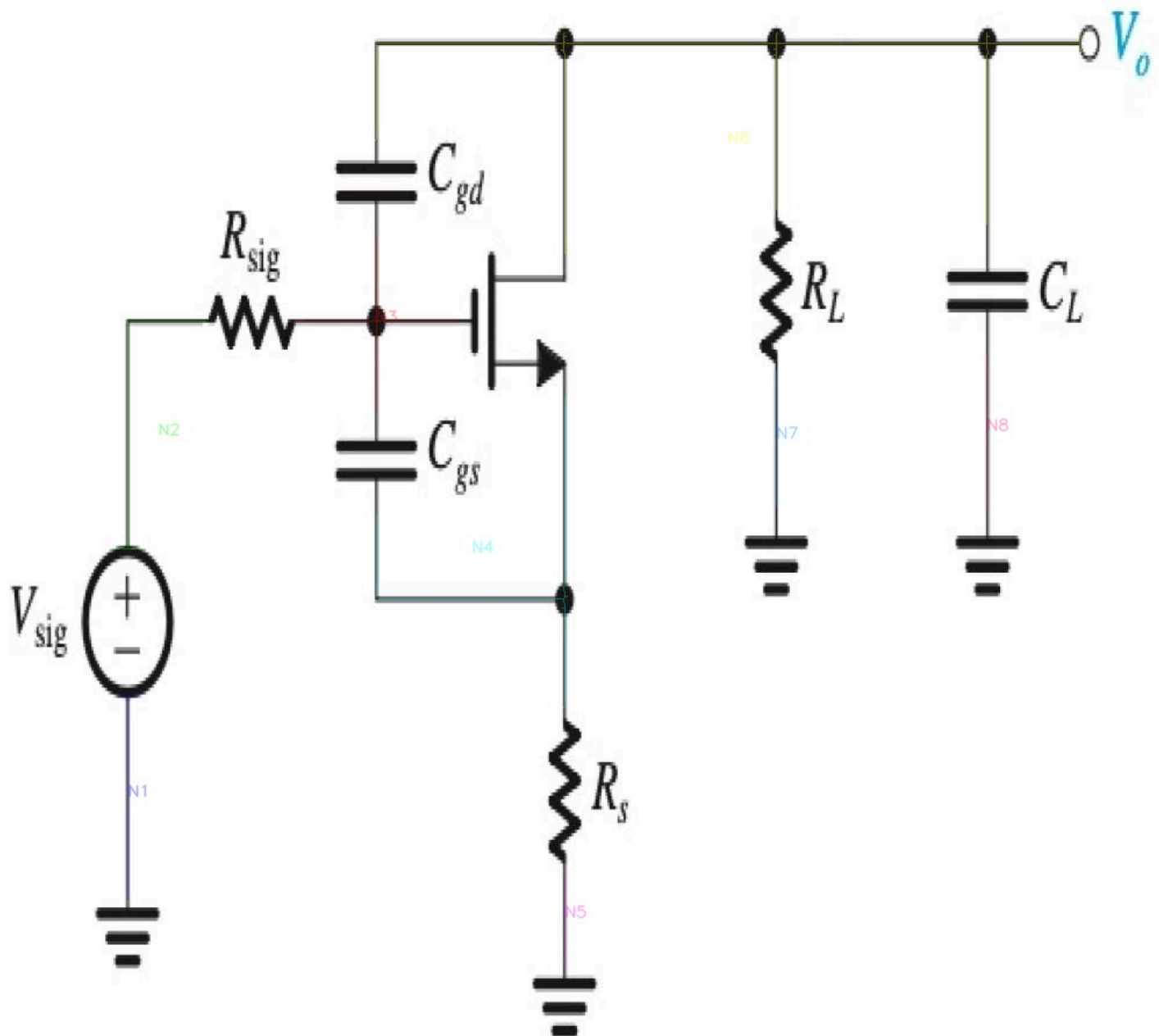


Figura 12. Overlay delle net ricostruite sull'immagine originale `pilot_001.jpg`. L'immagine mostra come i tratti di filo dello skeleton siano stati trasformati in net esplicite e localizzate sul diagramma.

9. Script `06_match_terminals_to_nets.py`

9.1 Obiettivo

Il sesto script della pipeline è:

```
scripts/topology/06_match_terminals_to_nets.py
```

Questo script ha il compito di:

- leggere il file JSON prodotto da `05_build_nets.py` ;
- leggere la `label_map` numerica delle net;
- verificare, per ogni terminale stimato, quale net è presente in prossimità del punto terminale;
- associare in modo esplicito ogni terminale a una net;
- aggiornare sia la lista globale dei terminali sia i terminali interni ai componenti;
- costruire una lista finale di connessioni terminale-net;
- salvare una nuova immagine di debug con i match visualizzati.

9.2 Logica generale

L'idea di questo blocco è trasformare in forma esplicita una relazione che nel blocco 05 era ancora implicita.

Nel blocco 05, infatti, era già noto che alcune net candidate risultavano vicine a determinati terminali. Lo script 06 formalizza questa informazione nella forma:

- `31.1:t1 -> N2`
- `31.1:t2 -> N1`
- `22.1:t1 -> N2`
- `22.1:t2 -> N3`
- ecc.

La procedura seguita è la seguente:

1. lettura del file JSON prodotto dal blocco 05;
2. caricamento della `label_map` delle net salvata come file `.npy` ;
3. costruzione di una mappa `net_index -> net` ;
4. per ogni terminale, analisi di una finestra circolare attorno alle coordinate `(x, y)` del terminale;
5. raccolta delle net candidate presenti in quella zona;

6. scelta della net corretta:

- se è presente una sola net candidata, il match è diretto;
- se sono presenti più net candidate, si seleziona quella con pixel più vicino al terminale;
- se non è presente alcuna net, il terminale viene marcato come `unmatched` ;

7. salvataggio del risultato nel JSON e in un'immagine di debug.

9.3 Strategia adottata

Per questa prima versione si è scelto un approccio semplice ma robusto:

- ricerca locale attorno al terminale con `match_radius = 8` ;
- eventuale raggio di fallback `fallback_radius = 16` ;
- assegnazione diretta quando è presente una sola net candidata;
- risoluzione per distanza minima nel caso di più candidate.

Questa strategia è stata ritenuta adeguata per il pilot, dato che la ricostruzione delle net ottenuta nel blocco 05 risultava già abbastanza pulita.

9.4 Output prodotti

Lo script produce e salva:

JSON con match terminale-net

`outputs/topology/06_match_terminals_to_nets/pilot_001.json`

Immagine debug con terminali associati alle net

`outputs/topology/06_match_terminals_to_nets/debug_images/pilot_001_terminal_net_matches.jpg`

Inoltre, il file JSON viene aggiornato con:

- la lista globale `connections` ;
- il numero totale di connessioni (`n_connections`);
- una sezione `terminal_net_matching` con informazioni di riepilogo.

9.5 Contenuto del JSON

Dopo l'esecuzione dello script, ogni terminale contiene campi aggiuntivi del tipo:

- `candidate_net_ids`
- `candidate_net_indices`
- `matched_net_id`
- `matched_net_index`
- `match_status`
- `match_distance_px`
- `used_radius`
- `used_fallback`

Viene inoltre creata una lista globale:

- `connections`

in cui ogni elemento rappresenta esplicitamente una relazione terminale-net.

Nel file risultante del pilot sono presenti **19 connessioni**, una per ciascun terminale stimato, e tutti i terminali risultano matchati con successo.

9.6 Risultato ottenuto su `pilot_001`

Nel caso dell'immagine pilota, il risultato del blocco 06 è particolarmente pulito:

- `n_terminals = 19`
- `n_matched_terminals = 19`
- `n_unmatched_terminals = 0`
- `n_connections = 19`

Inoltre, tutti i terminali risultano assegnati con stato:

- `match_status = matched_single`

cioè senza ambiguità e senza necessità di risoluzione complessa.

9.7 Esempi di associazioni ottenute

Alcuni esempi significativi sono:

- `31.1:t1 -> N2`
- `31.1:t2 -> N1`
- `22.1:t1 -> N2`
- `22.1:t2 -> N3`

- 4.2:t1 -> N6
- 4.2:t2 -> N3
- 22.3:t1 -> N6
- 22.3:t2 -> N7
- 4.3:t1 -> N6
- 4.3:t2 -> N8 :contentReference[oaicite:3]{index=3}

Queste associazioni sono coerenti con la struttura del diagramma pilota e confermano che la pipeline sta già producendo una prima rappresentazione circuitale consistente.

9.8 Interpretazione qualitativa del risultato

Il blocco 06 permette finalmente di leggere il circuito nel modo seguente:

- ogni **componente** ha uno o più **terminali**;
- ogni **terminale** appartiene a una specifica **net**.

Di conseguenza, per un componente a due terminali, è già possibile ricostruire una rappresentazione semplificata del tipo:

- Voltage_Source 31.1 tra N2 e N1
- Resistor 22.1 tra N2 e N3
- Capacitor 4.2 tra N6 e N3
- Capacitor 4.1 tra N3 e N4
- Resistor 22.2 tra N4 e N5
- Resistor 22.3 tra N6 e N7
- Capacitor 4.3 tra N6 e N8

Questa struttura costituisce, di fatto, una prima netlist semplificata del diagramma.

9.9 Osservazioni importanti

1. Tutti i terminali sono stati associati

Questo è un risultato molto positivo, perché mostra che:

- la stima dei terminali del blocco 03 è sufficientemente buona;
- l'estrazione dei fili del blocco 04 è sufficientemente stabile;
- la costruzione delle net del blocco 05 è abbastanza coerente per sostenere il matching finale.

2. Il risultato è ancora locale, non semantico

Il matching ottenuto è corretto dal punto di vista geometrico/topologico locale.

Non è ancora stata introdotta alcuna fusione semantica di simboli come `GND`, quindi net distinte che terminano su masse separate restano ancora distinte.

3. Il Mosfet resta escluso dalla topologia interna

Poiché il `Mosfet` non ha ancora terminali espliciti, il blocco 06 non ricostruisce ancora le connessioni che passano attraverso i suoi terminali.

Questa semplificazione resta coerente con la scelta progettuale iniziale del pilot.

9.10 Immagine di debug

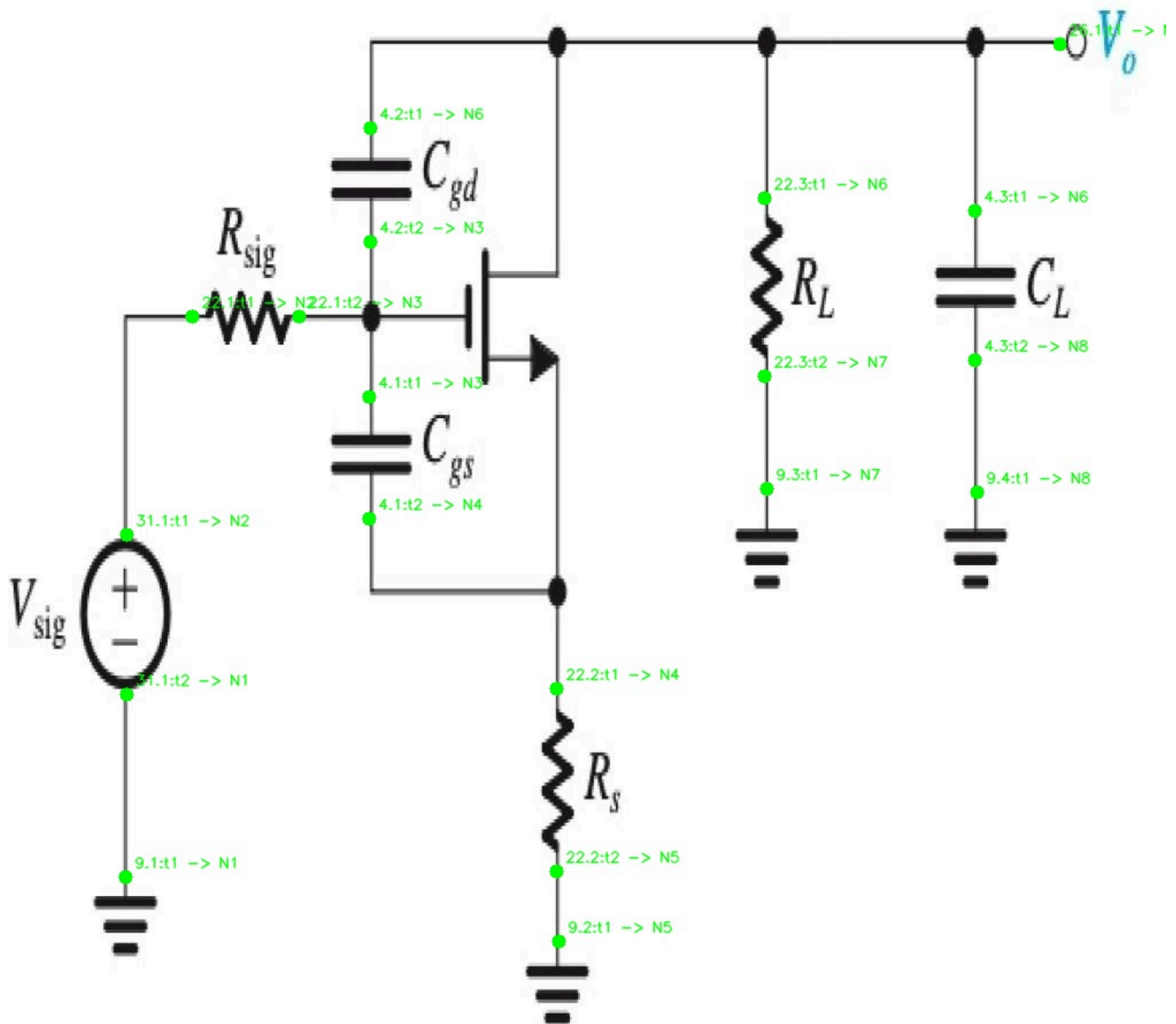


Figura 13. Risultato dello script `06_match_terminals_to_nets.py` su `pilot_001.jpg`. Ogni terminale stimato è mostrato insieme alla net a cui è stato associato. Nel caso dell'immagine pilota, tutti i 19 terminali risultano matchati correttamente a una net.

10. Script 07_export_graph.py

10.1 Obiettivo

Il settimo script della pipeline è:

```
scripts/topology/07_export_graph.py
```

Questo script ha il compito di:

- leggere il file JSON prodotto da 06_match_terminals_to_nets.py ;
- convertire componenti, terminali e net in una vera struttura a grafo;
- creare nodi di tipo:
 - Diagram
 - Component
 - Terminal
 - Net
- creare relazioni di tipo:
 - HAS_COMPONENT
 - HAS_NET
 - HAS_TERMINAL
 - CONNECTED_TO
- esportare il grafo sia in formato JSON sia in formato tabellare CSV.

10.2 Logica generale

L'idea di questo blocco è trasformare il risultato ottenuto nei passi precedenti in una struttura dati finale già compatibile con una rappresentazione a grafo.

La procedura seguita è la seguente:

1. lettura del file JSON prodotto dal blocco 06;
2. creazione di un nodo `Diagram` che rappresenta l'intero schema;
3. creazione di un nodo `Component` per ciascun componente rilevato;
4. creazione di un nodo `Terminal` per ciascun terminale stimato;
5. creazione di un nodo `Net` per ciascuna net ricostruita;
6. creazione degli archi:
 - `Diagram -> HAS_COMPONENT -> Component`

- Diagram -> HAS_NET -> Net
- Component -> HAS_TERMINAL -> Terminal
- Terminal -> CONNECTED_TO -> Net

7. esportazione del risultato in un file `graph.json` e in due file CSV separati per nodi e archi.

10.3 Motivazione della struttura scelta

La struttura a grafo scelta è coerente con la logica elettrica del problema.

In particolare, è preferibile rappresentare il circuito nel modo seguente:

- un componente possiede uno o più terminali;
- ogni terminale è collegato a una net;
- la net rappresenta il collegamento elettrico condiviso tra più terminali.

Questa impostazione è più corretta rispetto a un collegamento diretto componente-componente, perché mantiene esplicita la mediazione delle net.

10.4 Output prodotti

Lo script produce e salva:

JSON del grafo

`outputs/topology/07_export_graph/graph_json/pilot_001_graph.json`

CSV dei nodi

`outputs/topology/07_export_graph/nodes_csv/pilot_001_nodes.csv`

CSV degli archi

`outputs/topology/07_export_graph/edges_csv/pilot_001_edges.csv`

10.5 Contenuto del JSON

Il file `graph.json` contiene:

- `graph_metadata`

- graph_summary
- nodes
- edges

La sezione `nodes` contiene tutti i nodi del grafo, mentre la sezione `edges` contiene tutte le relazioni tra nodi.

Ogni nodo ha almeno:

- node_id
- node_type
- label
- diagram_id

Ogni arco ha almeno:

- edge_id
- source
- target
- relation_type
- diagram_id

10.6 Risultato ottenuto su pilot_001

Nel caso dell'immagine pilota, il grafo esportato contiene:

- 41 nodi totali;
- 59 archi totali.

Nel dettaglio:

- 1 nodo Diagram
- 13 nodi Component
- 19 nodi Terminal
- 8 nodi Net

e:

- 13 archi HAS_COMPONENT
- 8 archi HAS_NET
- 19 archi HAS_TERMINAL

- 19 archi `CONNECTED_TO`

Questo risultato è coerente con tutti i blocchi precedenti della pipeline e rappresenta la prima versione completa del grafo del diagramma.

10.7 Interpretazione qualitativa del risultato

Il grafo finale consente già di rappresentare il diagramma nel modo seguente:

- lo schema `pilot_001` è un nodo `Diagram` ;
- i componenti rilevati sono nodi `Component` ;
- i terminali stimati sono nodi `Terminal` ;
- le reti ricostruite sono nodi `Net` ;
- ogni componente possiede i propri terminali;
- ogni terminale è collegato a una specifica net.

Di conseguenza, il circuito non è più rappresentato soltanto come immagine, ma come struttura logica interrogabile.

10.8 Esempi di relazioni presenti nel grafo

Nel grafo risultante compaiono relazioni del tipo:

- `diagram:pilot_001 -> component:31.1` con relazione `HAS_COMPONENT`
- `component:22.1 -> terminal:22.1:t1` con relazione `HAS_TERMINAL`
- `terminal:22.1:t1 -> net:N2` con relazione `CONNECTED_TO`
- `terminal:22.1:t2 -> net:N3` con relazione `CONNECTED_TO`

Questo significa che il grafo finale conserva sia la struttura gerarchica del diagramma sia la connettività elettrica locale ricostruita nei blocchi precedenti.

10.9 Osservazioni importanti

1. Il grafo è coerente con il pilot

I conteggi dei nodi e degli archi sono perfettamente consistenti con:

- i componenti rilevati;
- i terminali stimati;

- le net costruite;
- le connessioni terminale-net associate nei blocchi precedenti.

2. Il Mosfet è presente come componente ma non ancora come nodo topologico completo

Il Mosfet compare correttamente come nodo `Component`, ma non avendo ancora terminali espliciti non genera nodi `Terminal` né relazioni `CONNECTED_TO`.

3. Le masse non sono ancora fuse semanticamente

Le net che terminano su simboli `GND` distinti restano separate nel grafo, perché la pipeline attuale non applica ancora una fusione semantica della massa.

10.10 Valore del risultato

Con questo blocco si conclude con successo un primo MVP completo della pipeline:

immagine → componenti → istanze → terminali → fili → net → associazioni terminale-net → grafo

Questo risultato rappresenta il primo passaggio concreto dal dominio visivo del diagramma al dominio strutturato e interrogabile del graph modeling.